

UNIVERSIDADE FEDERAL DO AMAZONAS

Instituto de Computação

ICC205 – INTRODUÇÃO A BANCOS DE DADOS

TRABALHO PRÁTICO II

Armazenamento em Memória Secundária,
Hashing e Índices B+Tree

Aluno: Cirus Alexandre Tinga
Professor: Altigran Soares da Silva

Manaus – AM
2026

Sumário

1 Introdução **2**

2 Decisões de Projeto **3**
2.1 Representação dos registros . . . 3
2.2 Organização do arquivo de dados . . . 3
2.3 Endereçamento dos registros . . . 4
2.4 Índice B+Tree por identificador . . . 4
2.5 Índice B+Tree por título . . . 4

3 Previsões Antes das Medições **4**
3.1 Busca por identificador . . . 5
3.2 Busca por título . . . 5
3.3 Busca por intervalo de identificadores . . . 5
3.4 Efeito do tamanho do bloco . . . 5
3.5 Resumo das previsões . . . 6

4 Experimentos com Blocos de 4096 Bytes **6**
4.1 Busca por identificador . . . 7
4.2 Busca por título . . . 8
4.3 Busca por intervalo . . . 9
4.4 Título duplicado . . . 10

5 Comparação entre Previsões e Resultados **11**
5.1 Busca por identificador . . . 11
5.2 Busca por título . . . 12
5.3 Busca por intervalo . . . 12
5.4 Análise geral . . . 12

6 Experimentos com Blocos de 8192 Bytes **12**
6.1 Carga dos dados . . . 13
6.2 Busca por identificador . . . 14
6.3 Busca por título . . . 14
6.4 Busca por intervalo . . . 15
6.5 Análise do impacto do tamanho do bloco . . . 16

7 Casos Patológicos **16**
7.1 Identificador inexistente . . . 16
7.2 Título inexistente . . . 16
7.3 Intervalo sem resultados . . . 16
7.4 Título duplicado . . . 16
7.5 Resumo dos casos testados . . . 17

8 Declaração de Uso de Inteligência Artificial **17**

9 Conclusão **19**

1 Introdução

Este trabalho tem como objetivo implementar e analisar técnicas de organização e acesso a dados em memória secundária. A base utilizada é composta por registros de artigos lidos de um arquivo CSV e armazenados em um arquivo paginado.

Na primeira parte, o arquivo de dados foi organizado por hashing do identificador dos artigos. Sobre essa organização foram implementados os programas `upload`, `findrec`, `findtitle` e `findrange`.

Na segunda parte, foram construídos dois índices B+Tree: um índice primário por identificador e um índice secundário por título. Esses índices são utilizados pelos programas `seek1`, `seek2` e `seekrange`.

Os experimentos comparam a quantidade de blocos lidos pelas operações sem índice e pelas operações equivalentes utilizando B+Tree. Também é analisado o impacto da mudança do tamanho do bloco de 4096 para 8192 bytes.

2 Decisões de Projeto

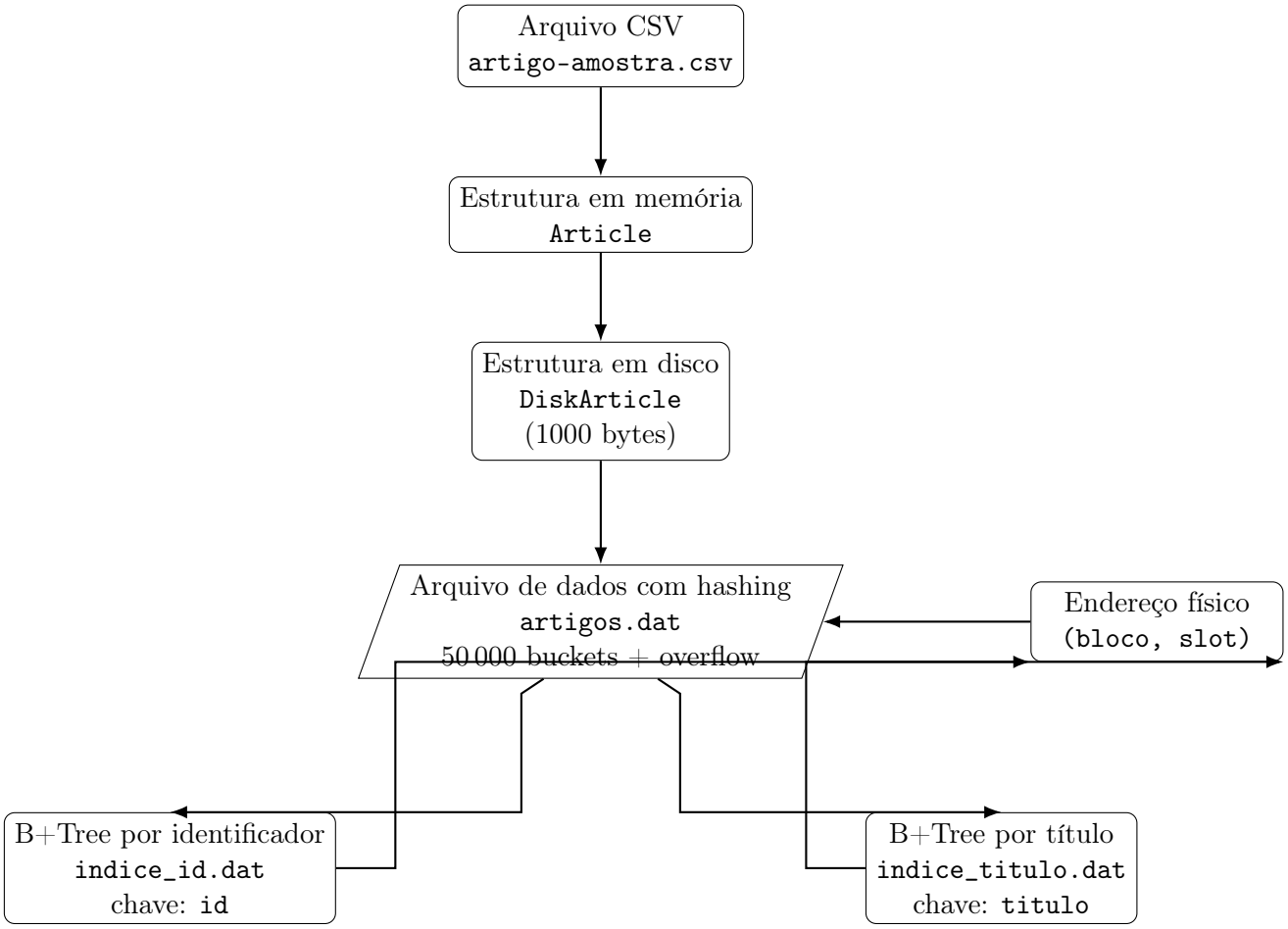


Figura 1: Visão geral da organização do arquivo de dados e dos índices.

2.1 Representação dos registros

Os registros lidos do arquivo CSV são inicialmente representados pela estrutura `Article`, fornecida no projeto.

Para o armazenamento em memória secundária foi utilizada uma estrutura de tamanho fixo denominada `DiskArticle`.

O tamanho obtido para cada registro em disco foi de 1000 bytes.

Os campos textuais foram armazenados utilizando vetores de caracteres de tamanho fixo. Essa escolha simplifica o posicionamento dos registros dentro dos blocos e a recuperação posterior por meio dos índices.

O campo `snippet` utiliza espaço limitado, podendo ocorrer truncamento quando o conteúdo ultrapassa a capacidade definida.

2.2 Organização do arquivo de dados

O arquivo de dados foi organizado utilizando hashing.

Foram utilizados 50 000 buckets e a função de dispersão:

$$h(id) = id \bmod 50000$$

Cada bucket possui inicialmente um bloco principal.

Com blocos de 4096 bytes e registros de 1000 bytes, são armazenados até quatro registros por bloco.

Quando um bloco fica cheio, um novo bloco de overflow é alocado e ligado ao anterior através de um ponteiro armazenado no cabeçalho do bloco.

2.3 Endereçamento dos registros

Cada registro armazenado no arquivo de dados pode ser identificado por um endereço físico composto por:

- número do bloco;
- posição do registro dentro do bloco (*slot*).

Esse endereço é utilizado pelos índices B+Tree para localizar os registros no arquivo de dados.

2.4 Índice B+Tree por identificador

Foi implementada uma B+Tree cuja chave é o identificador do artigo.

As folhas armazenam pares da forma:

$$(id,\ bloco,\ slot)$$

Os nós internos armazenam as chaves de separação e os ponteiros para os nós filhos.

As folhas são encadeadas, permitindo a execução eficiente de consultas por intervalo de identificadores.

2.5 Índice B+Tree por título

Também foi implementado um índice secundário utilizando o título do artigo como chave.

Como podem existir diferentes artigos com o mesmo título, o índice permite armazenar múltiplas entradas para uma mesma chave.

Durante a busca, todas as entradas correspondentes ao título solicitado são recuperadas e seus endereços são utilizados para acessar o arquivo de dados.

3 Previsões Antes das Medições

Antes da execução dos experimentos, foram estabelecidas previsões sobre a quantidade de blocos acessados por cada tipo de consulta.

O objetivo desta etapa é comparar posteriormente o comportamento esperado com os valores efetivamente medidos durante a execução dos programas.

3.1 Busca por identificador

O programa `findrec` utiliza diretamente a organização por hashing do arquivo de dados.

A partir do identificador do artigo é calculado o bucket correspondente pela função:

$$h(id) = id \bmod 50000$$

Dessa forma, espera-se que a busca por identificador necessite de poucos acessos a blocos, normalmente apenas ao bloco principal do bucket e, quando necessário, aos blocos de overflow.

Já o programa `seek1` utiliza a B+Tree de identificadores. Espera-se que sejam lidos alguns blocos do índice durante a navegação da raiz até uma folha e, após a localização do endereço do registro, apenas um bloco do arquivo de dados.

Assim, para consultas pontuais por identificador, espera-se que tanto o hashing quanto a B+Tree apresentem baixo custo de acesso.

3.2 Busca por título

O programa `findtitle` não possui um mecanismo de acesso direto pelo título.

Como o arquivo de dados está organizado por hashing do identificador, a busca por título deve percorrer sequencialmente os blocos até encontrar o registro desejado.

Caso o título não exista, espera-se que todos os blocos do arquivo sejam lidos.

Por outro lado, o programa `seek2` utiliza uma B+Tree secundária por título.

Espera-se que a busca percorra apenas os níveis da árvore necessários para alcançar a folha correspondente e, depois, acesse somente os blocos do arquivo de dados que contêm os registros encontrados.

Portanto, espera-se uma redução significativa na quantidade de blocos lidos com o uso do índice por título.

3.3 Busca por intervalo de identificadores

O programa `findrange` precisa realizar uma varredura completa do arquivo de dados.

Isso ocorre porque a organização por hashing não preserva a ordenação dos identificadores.

Assim, mesmo para intervalos pequenos, espera-se que todos os blocos do arquivo sejam examinados.

O programa `seekrange`, por outro lado, utiliza a ordenação das chaves na B+Tree.

A busca deve localizar inicialmente a folha contendo o limite inferior do intervalo e, em seguida, percorrer as folhas encadeadas até ultrapassar o limite superior.

Dessa forma, espera-se que `seekrange` realize uma quantidade muito menor de leituras que `findrange`, principalmente para intervalos pequenos.

3.4 Efeito do tamanho do bloco

Também será realizado um experimento com blocos de 8192 bytes.

Como o aumento do tamanho do bloco permite armazenar uma quantidade maior de registros ou chaves em cada página, espera-se uma redução no número total de blocos utilizados pelos arquivos.

Nas B+Trees, espera-se também um aumento do fator de ramificação, o que pode reduzir a quantidade de níveis da árvore e, consequentemente, o número de blocos acessados durante uma consulta.

Entretanto, os valores exatos somente serão conhecidos após a execução dos experimentos.

3.5 Resumo das previsões

Tabela 1: Previsão do comportamento das operações.

Operação	**Estratégia**	**Comportamento esperado**
`findrec`	Hashing	Poucos blocos, podendo aumentar em caso de overflow.
`seek1`	B+Tree por ID	Poucos blocos do índice e um bloco de dados.
`findtitle`	Varredura sequencial	Pode exigir grande quantidade de blocos e, no pior caso, todo o arquivo.
`seek2`	B+Tree por título	Poucos blocos do índice e apenas os blocos dos registros encontrados.
`findrange`	Varredura sequencial	Leitura de todo o arquivo de dados.
`seekrange`	B+Tree por ID	Busca inicial pela árvore e percurso das folhas necessárias.

4 Experimentos com Blocos de 4096 Bytes

Nesta etapa foram executadas as consultas utilizando blocos de 4096 bytes. Os experimentos comparam as consultas sem uso de índice com as consultas equivalentes utilizando índices B+Tree.

Durante a carga foram processados 204285 registros, sendo duas linhas ignoradas pelo leitor do arquivo CSV. Cada registro armazenado em disco possui 1000 bytes e, com blocos de 4096 bytes, são armazenados até quatro registros por bloco.

O arquivo de dados utilizou 69940 blocos. O índice B+Tree por identificador utilizou 2961 blocos, enquanto o índice B+Tree por título utilizou 33208 blocos. A taxa aproximada de ocupação do arquivo de dados foi de 71,31%.

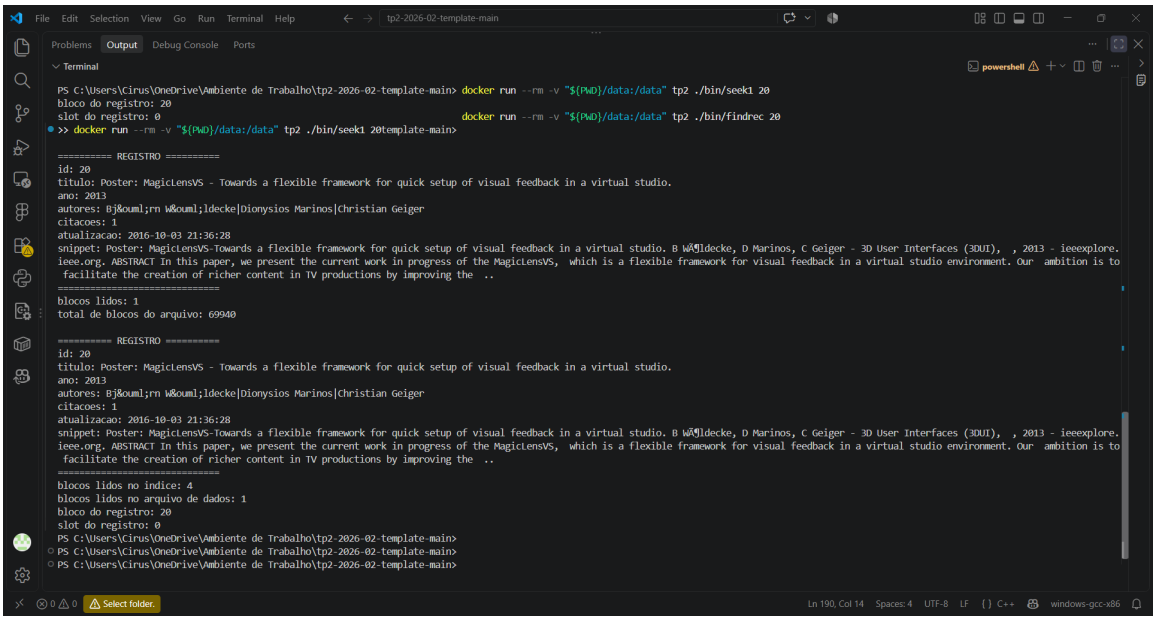


Figura 2: Carga da base utilizando blocos de 4096 bytes.

4.1 Busca por identificador

Foi utilizado o identificador 20, correspondente a um registro existente no conjunto de dados.

O programa `findrec` realiza a busca diretamente no arquivo organizado por hashing. Para esse registro foi necessário acessar apenas um bloco do arquivo de dados.

O programa `seek1` utiliza a B+Tree por identificador. Foram necessárias quatro leituras no índice até alcançar a folha correspondente e uma leitura adicional no arquivo de dados.

Tabela 2: Busca por identificador com blocos de 4096 bytes.

Programa	Blocos no índice	Blocos nos dados
`findrec`	–	1
`seek1`	4	1

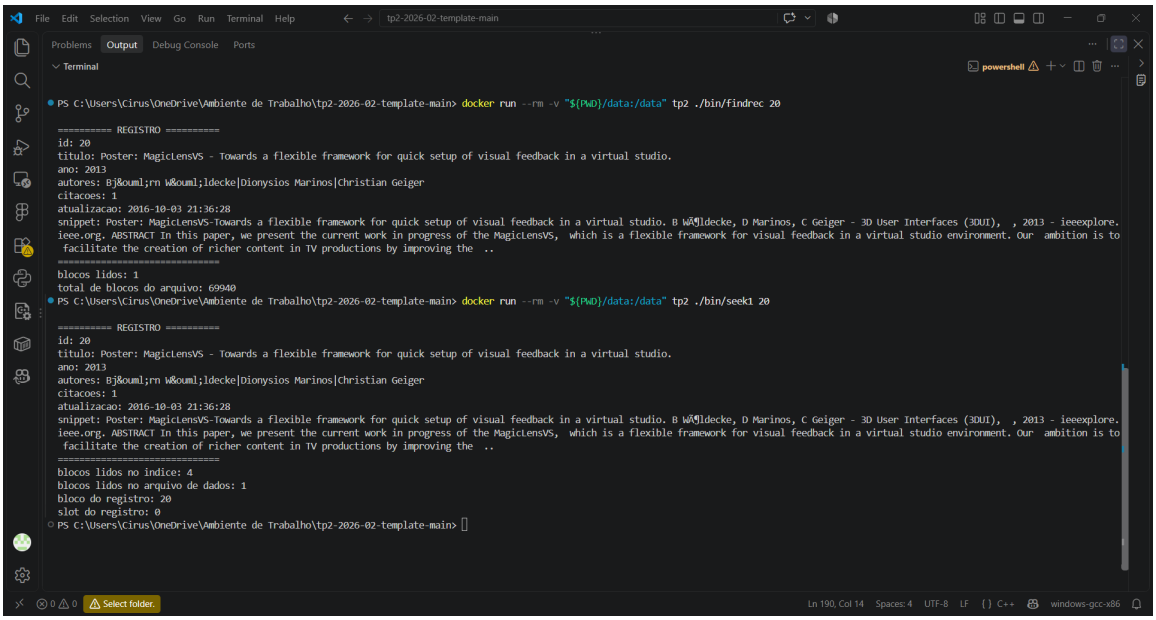


Figura 3: Comparação entre `findrec` e `seek1` com blocos de 4096 bytes.

Nesse caso, o hashing apresentou menor quantidade de leituras, pois o identificador é justamente o atributo utilizado pela função de dispersão, permitindo acesso direto ao bucket correspondente.

4.2 Busca por título

Para a busca por título foi utilizado o registro de identificador 20, cujo título é:

> *Poster: MagicLensVS - Towards a flexible framework for quick setup of visual feedback in a virtual studio.*

O programa `findtitle` realizou uma busca sequencial e encontrou o registro após a leitura de 21 blocos. Já o programa `seek2`, utilizando o índice B+Tree secundário, necessitou de oito leituras no índice e apenas uma leitura no arquivo de dados.

Tabela 3: Busca por título com blocos de 4096 bytes.

Programa	Blocos no índice	Blocos nos dados
`findtitle`	–	21
`seek2`	8	1

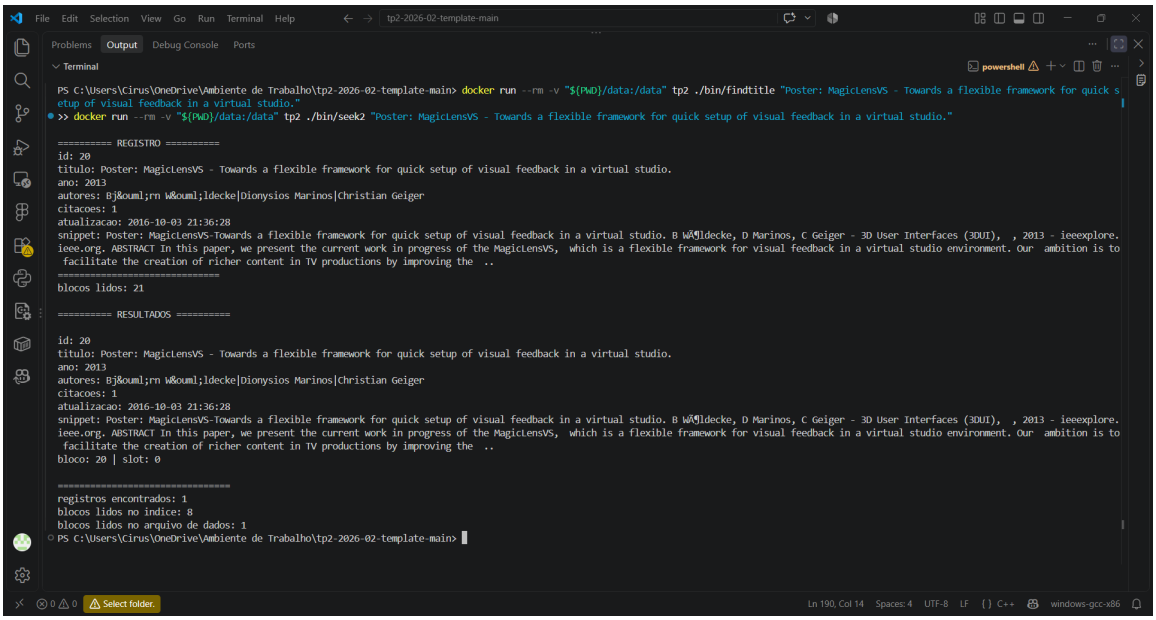


Figura 4: Comparação entre `findtitle` e `seek2` com blocos de 4096 bytes.

O resultado mostra a vantagem do índice quando a consulta é realizada por um atributo diferente daquele utilizado na organização hash do arquivo.

4.3 Busca por intervalo

Foi utilizado o intervalo de identificadores de 20 a 30. Foram encontrados três registros, correspondentes aos identificadores 20, 25 e 30.

Como o hashing não mantém os registros ordenados, o programa `findrange` precisou examinar todos os 69940 blocos do arquivo de dados. O programa `seekrange` utilizou a B+Tree para localizar o início do intervalo, realizando quatro leituras no índice e três leituras no arquivo de dados.

Tabela 4: Busca por intervalo com blocos de 4096 bytes.

Programa	Blocos no índice	Blocos nos dados
`findrange`	–	69940
`seekrange`	4	3

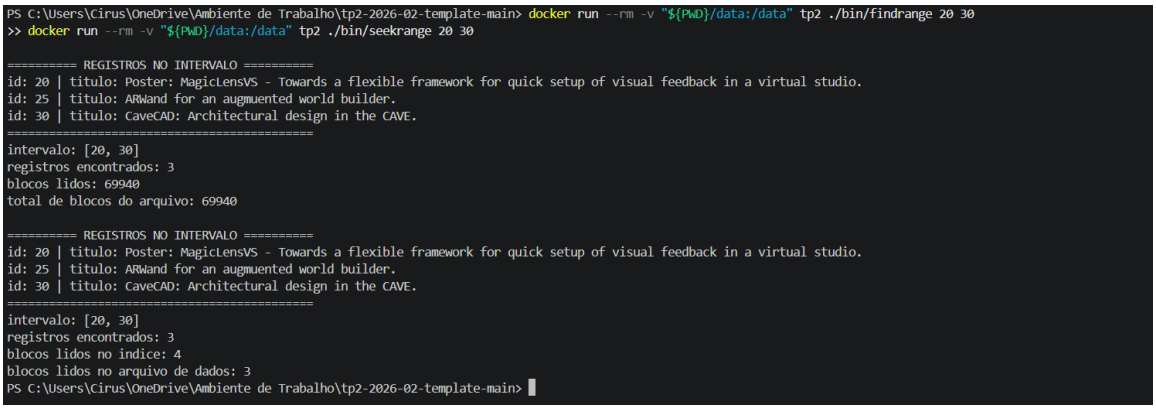


Figura 5: Comparação entre `findrange` e `seekrange` para o intervalo de 20 a 30.

A diferença foi significativa, pois a consulta indexada evitou a varredura completa do arquivo.

4.4 Título duplicado

Também foi realizado um teste utilizando o título:

Spiteful Bidding in the Dollar Auction.

Esse título aparece duas vezes no conjunto de dados. O programa `seek2` recuperou corretamente os dois registros, com identificadores 131223 e 2788. Foram realizadas oito leituras no índice e duas leituras no arquivo de dados.

Tabela 5: Busca por título duplicado com `seek2`.

Métrica	**Resultado**
Registros encontrados	2
Blocos lidos no índice	8
Blocos lidos nos dados	2

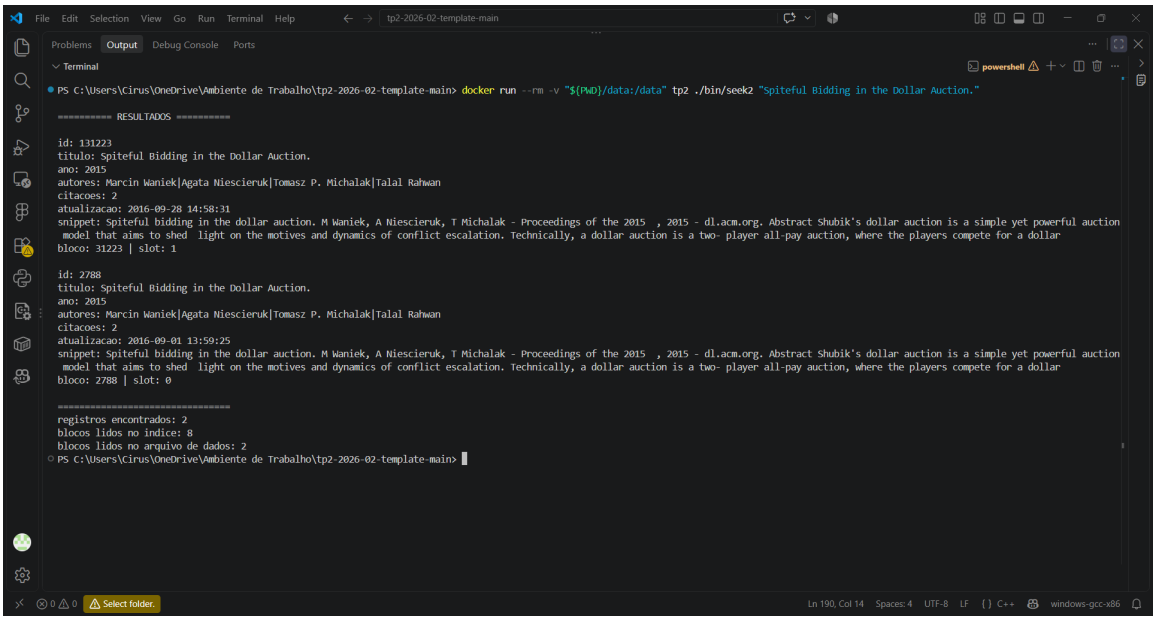


Figura 6: Recuperação de dois registros associados ao mesmo título utilizando `seek2`.

O teste confirma que o índice secundário implementado é capaz de tratar chaves duplicadas e manter diferentes endereços físicos associados ao mesmo título.

5 Comparação entre Previsões e Resultados

Os resultados obtidos com blocos de 4096 bytes permitem comparar as previsões realizadas anteriormente com as medições efetivamente observadas.

De forma geral, o comportamento previsto foi confirmado. O hashing apresentou baixo custo para consultas pontuais pelo identificador, enquanto os índices B+Tree apresentaram maior vantagem nas consultas por título e por intervalo.

5.1 Busca por identificador

Para o identificador 20, o programa `findrec` realizou somente uma leitura no arquivo de dados. O programa `seek1` realizou quatro leituras no índice e uma leitura no arquivo de dados, totalizando cinco leituras.

Tabela 6: Comparação da busca por identificador.

Programa	Índice	Dados	Total de leituras
`findrec`	–	1	1
`seek1`	4	1	5

Nesse caso, o hashing apresentou vantagem, confirmando a previsão de que uma busca pontual pela própria chave de dispersão possui baixo custo.

5.2 Busca por título

Na busca pelo título do registro 20, o programa `findtitle` realizou 21 leituras no arquivo de dados. O programa `seek2` realizou oito leituras no índice e uma leitura no arquivo de dados, totalizando nove leituras.

Tabela 7: Comparação da busca por título.

Programa	Índice	Dados	Total de leituras
`findtitle`	–	21	21
`seek2`	8	1	9

A utilização do índice reduziu a quantidade total de leituras de 21 para 9 nesse caso.

5.3 Busca por intervalo

Para o intervalo de identificadores de 20 a 30, o programa `findrange` realizou a leitura dos 69940 blocos do arquivo. Já o `seekrange` realizou apenas quatro leituras no índice e três leituras nos dados, totalizando sete blocos.

Tabela 8: Comparação da busca por intervalo.

Programa	Índice	Dados	Total de leituras
`findrange`	–	69940	69940
`seekrange`	4	3	7

Esse resultado confirma a principal vantagem da B+Tree para consultas por intervalo. Como as chaves estão ordenadas e as folhas estão encadeadas, não é necessário realizar uma varredura completa do arquivo.

5.4 Análise geral

Os experimentos confirmaram as previsões estabelecidas antes das medições. Para busca pontual por identificador, o hashing apresentou o menor custo, com apenas uma leitura no caso testado. A utilização da B+Tree adicionou o custo de navegação pelo índice.

Na busca por título, a B+Tree secundária reduziu a quantidade de leituras ao permitir localizar diretamente o endereço do registro. A maior diferença ocorreu na consulta por intervalo: a busca sequencial necessitou ler 69940 blocos, enquanto a consulta utilizando a B+Tree necessitou de apenas sete leituras no total.

Portanto, os experimentos mostram que a estrutura mais adequada depende do tipo de consulta. O hashing é particularmente eficiente para busca pontual pela chave utilizada na função de dispersão, enquanto a B+Tree apresenta vantagens importantes para índices secundários e consultas por intervalo.

6 Experimentos com Blocos de 8192 Bytes

Após os experimentos com blocos de 4096 bytes, o projeto foi recompilado utilizando blocos de 8192 bytes. O objetivo desta etapa foi avaliar o impacto do aumento do tamanho

do bloco sobre o arquivo de dados, os índices e a quantidade de leituras realizadas pelas consultas.

A compilação foi realizada utilizando a definição:

```
BLOCK_SIZE=8192
```

6.1 Carga dos dados

A base foi carregada novamente utilizando blocos de 8192 bytes. Foram processados 204285 registros, mantendo o mesmo conjunto utilizado no experimento com blocos de 4096 bytes.

Com o aumento do tamanho do bloco, a capacidade passou de quatro para oito registros por bloco. O arquivo de dados passou de 69940 para 51134 blocos. Entretanto, a taxa aproximada de ocupação caiu de 71,31% para 48,77%, pois os 50000 buckets principais continuam sendo previamente reservados e muitos deles não utilizam toda a capacidade adicional do bloco maior.

Tabela 9: Comparação da carga com diferentes tamanhos de bloco.

Métrica	**4096 bytes**	**8192 bytes**
Registros carregados	204285	204285
Registros por bloco	4	8
Blocos do arquivo de dados	69940	51134
Blocos do índice por ID	2961	2961
Blocos do índice por título	33208	33208
Taxa de ocupação	71,31%	48,77%

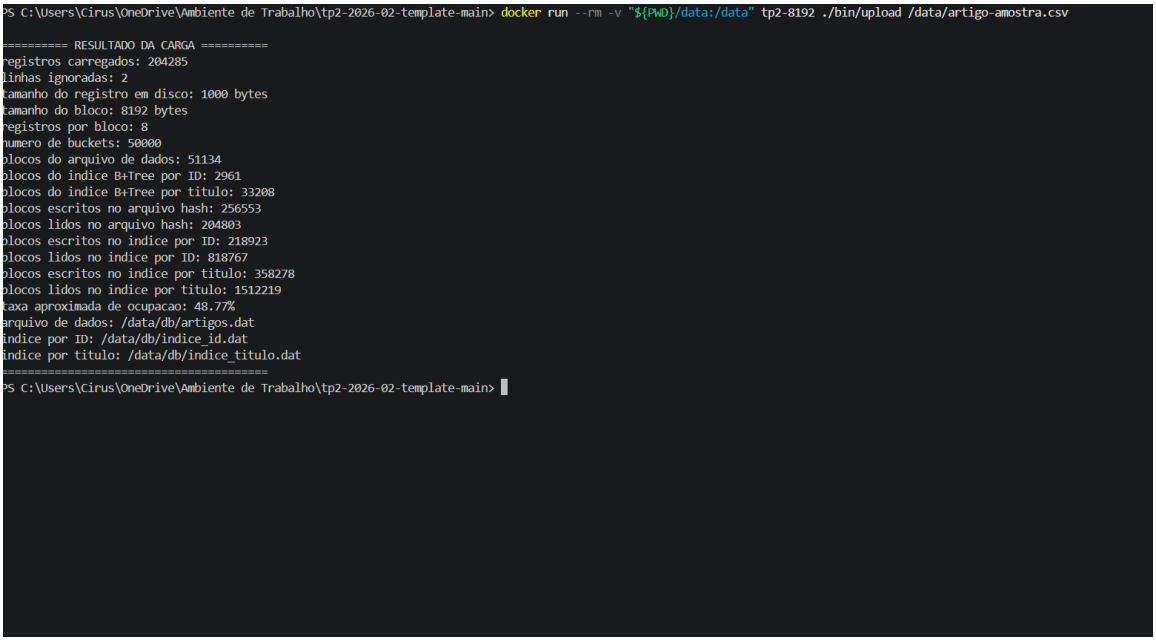


Figura 7: Carga da base utilizando blocos de 8192 bytes.

6.2 Busca por identificador

Os programas `findrec` e `seek1` foram novamente executados utilizando o identificador 20.

O `findrec` continuou necessitando de apenas uma leitura no arquivo de dados. O `seek1` realizou quatro leituras no índice e uma leitura no arquivo de dados, totalizando cinco leituras, o mesmo comportamento observado com blocos de 4096 bytes.

Tabela 10: Busca por identificador com blocos de 8192 bytes.

Programa	Blocos no índice	Blocos nos dados
`findrec`	–	1
`seek1`	4	1

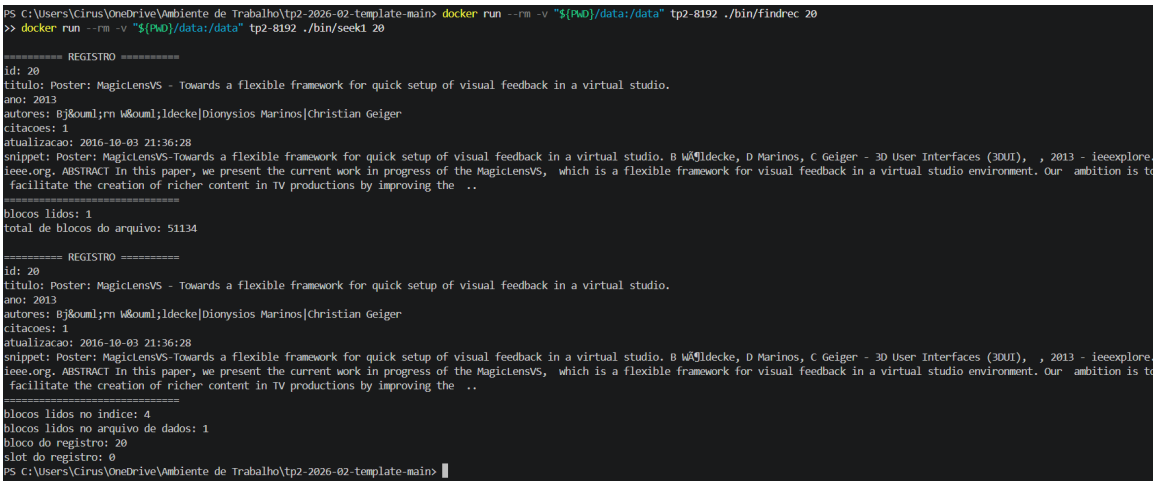


Figura 8: Comparação entre `findrec` e `seek1` com blocos de 8192 bytes.

6.3 Busca por título

Também foram repetidas as consultas por título utilizando `findtitle` e `seek2`. Para o mesmo título utilizado anteriormente, o `findtitle` realizou 21 leituras no arquivo de dados. O `seek2` realizou oito leituras no índice e uma leitura no arquivo de dados.

Tabela 11: Busca por título com blocos de 8192 bytes.

Programa	Blocos no índice	Blocos nos dados
`findtitle`	–	21
`seek2`	8	1

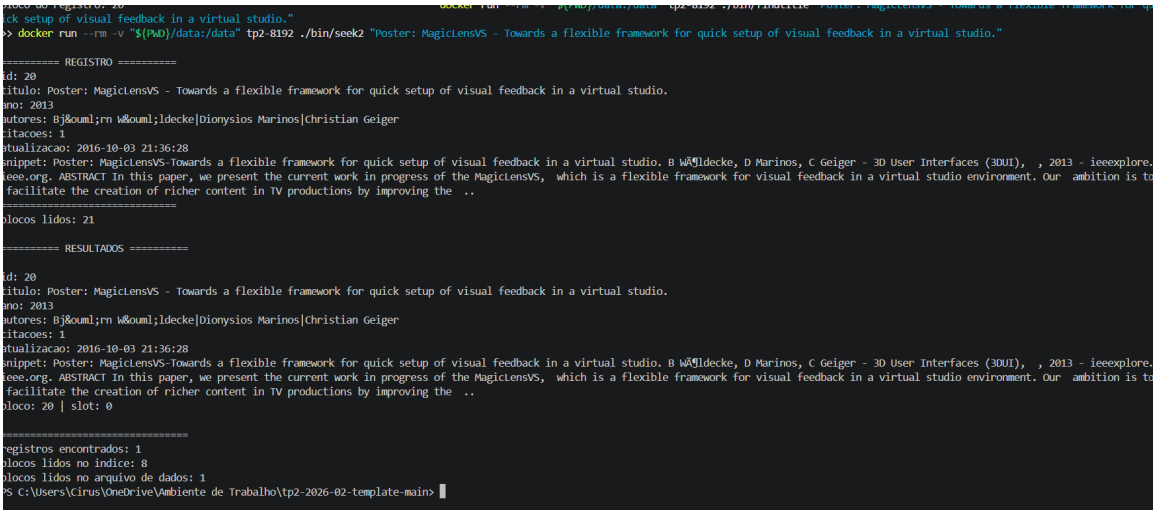


Figura 9: Comparação entre `findtitle` e `seek2` com blocos de 8192 bytes.

6.4 Busca por intervalo

Os programas `findrange` e `seekrange` foram executados novamente utilizando o intervalo de 20 a 30. Os dois programas encontraram os mesmos três registros, com identificadores 20, 25 e 30.

O `findrange` precisou percorrer os 51134 blocos do arquivo de dados. Esse valor é menor que os 69940 blocos lidos no experimento com páginas de 4096 bytes, refletindo a redução do número total de blocos do arquivo.

O `seekrange` realizou quatro leituras no índice e três leituras no arquivo de dados, totalizando sete leituras.

Tabela 12: Busca por intervalo com blocos de 8192 bytes.

Programa	Blocos no índice	Blocos nos dados
`findrange`	–	51134
`seekrange`	4	3

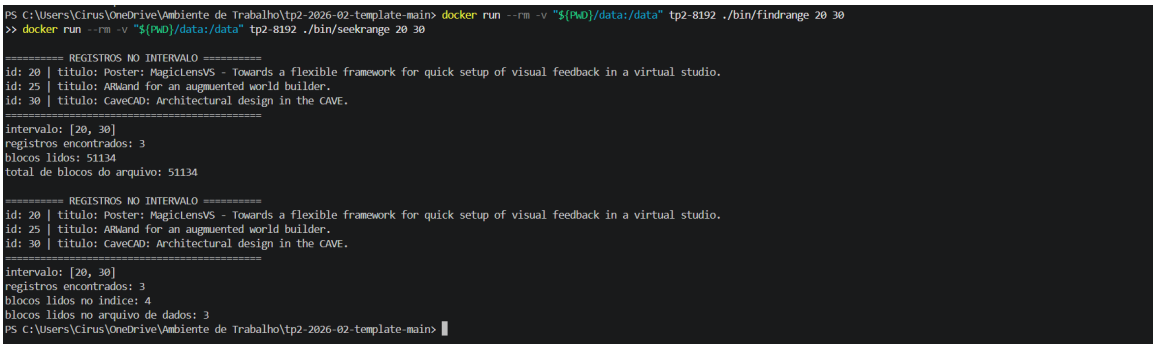


Figura 10: Comparação entre `findrange` e `seekrange` com blocos de 8192 bytes.

6.5 Análise do impacto do tamanho do bloco

O aumento do tamanho do bloco reduziu o número de blocos necessários para o arquivo de dados de 69940 para 51134. A principal consequência observada foi a redução do custo das operações que precisam percorrer o arquivo inteiro, como o `findrange`.

Por outro lado, as buscas pontuais apresentaram os mesmos números de leituras nas duas configurações. O `findrec` continuou com uma leitura, o `seek1` com quatro leituras de índice e uma de dados, e o `seek2` com oito leituras de índice e uma de dados.

Os índices B+Tree também mantiveram a mesma quantidade de blocos nas duas configurações: 2961 blocos no índice por identificador e 33208 blocos no índice por título. Isso ocorreu porque, nesta implementação, a capacidade máxima dos nós das árvores foi definida por constantes e não aumentou automaticamente com o valor de `BLOCK_SIZE`. Assim, o bloco físico maior reduziu o número de páginas do arquivo hash, mas não alterou a organização lógica das B+Trees.

A previsão de que blocos maiores reduziriam o número de páginas foi confirmada para o arquivo de dados. Já a previsão de redução da altura ou da quantidade de páginas das B+Trees não se confirmou nesta implementação específica.

7 Casos Patológicos

Além das consultas convencionais, foram realizados testes específicos para avaliar o comportamento das estruturas em situações especiais.

7.1 Identificador inexistente

Foi utilizado um identificador inexistente no conjunto de dados. O programa `findrec` realizou duas leituras no arquivo de dados antes de concluir que o registro não existia. O programa `seek1` realizou quatro leituras no índice B+Tree e nenhuma leitura no arquivo de dados.

7.2 Título inexistente

Foi realizada uma consulta utilizando um título inexistente. O programa `findtitle` precisou percorrer todo o arquivo de dados, enquanto o programa `seek2` realizou sete leituras no índice e nenhuma leitura no arquivo de dados.

7.3 Intervalo sem resultados

Também foi utilizado um intervalo de identificadores sem registros. O programa `findrange` realizou uma varredura completa do arquivo. O programa `seekrange` realizou quatro leituras no índice e nenhuma leitura no arquivo de dados.

7.4 Título duplicado

O índice secundário por título foi testado utilizando:

> *Spiteful Bidding in the Dollar Auction.*

Foram encontrados dois registros diferentes, com identificadores 131223 e 2788. A consulta realizou oito leituras no índice e duas leituras no arquivo de dados.

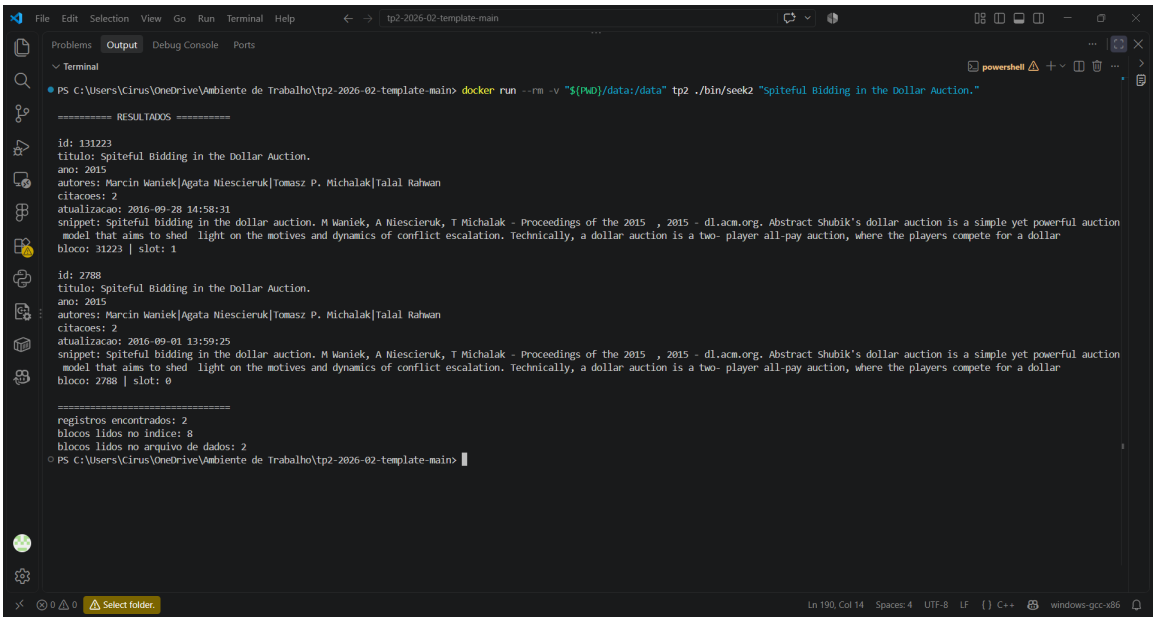


Figura 11: Consulta de título duplicado utilizando `seek2`.

7.5 Resumo dos casos testados

Tabela 13: Resultados dos casos especiais.

Caso	Programa	Índice	Dados
ID inexistente	`findrec`	–	2
ID inexistente	`seek1`	4	0
Título inexistente	`findtitle`	–	69940
Título inexistente	`seek2`	7	0
Intervalo vazio	`findrange`	–	69940
Intervalo vazio	`seekrange`	4	0
Título duplicado	`seek2`	8	2

8 Declaração de Uso de Inteligência Artificial

Durante o desenvolvimento deste trabalho foi utilizado o ChatGPT como ferramenta de apoio ao processo de implementação e documentação.

A ferramenta foi utilizada principalmente para:

- auxiliar na interpretação do enunciado;
- sugerir estruturas de organização para o código;
- apoiar a implementação das estruturas de hashing e B+Tree;
- identificar e corrigir erros de compilação;

- auxiliar na elaboração de casos de teste;
- organizar os resultados experimentais;
- apoiar a redação e estruturação deste relatório.

As decisões de projeto, a execução dos programas, a compilação, os testes e a validação dos resultados foram realizados durante o desenvolvimento do trabalho. Os códigos sugeridos foram compilados e testados no ambiente definido pelo projeto, com as correções necessárias realizadas conforme os resultados obtidos.

9 Conclusão

O desenvolvimento deste trabalho permitiu analisar, na prática, diferentes formas de organização e acesso a dados armazenados em memória secundária.

A organização por hashing apresentou bom desempenho nas buscas pontuais por identificador. No teste com o identificador 20, o `findrec` necessitou de apenas uma leitura de bloco. Por outro lado, a B+Tree por identificador precisou de quatro leituras no índice e uma leitura no arquivo de dados.

Nas consultas por título e por intervalo, os índices mostraram maior vantagem. Para o título utilizado no experimento, a quantidade total de leituras caiu de 21 no `findtitle` para 9 no `seek2`. Na busca pelo intervalo de 20 a 30, com blocos de 4096 bytes, o `findrange` leu 69940 blocos, enquanto o `seekrange` realizou apenas sete leituras no total.

O índice secundário por título também tratou corretamente chaves duplicadas, recuperando dois registros distintos associados ao título *Spiteful Bidding in the Dollar Auction.*.

O experimento com blocos de 8192 bytes mostrou que o aumento da página reduziu o arquivo de dados de 69940 para 51134 blocos e elevou a capacidade de quatro para oito registros por bloco. Como consequência, a varredura realizada pelo `findrange` caiu de 69940 para 51134 leituras. Entretanto, as consultas indexadas mantiveram os mesmos custos observados com 4096 bytes, pois a capacidade lógica dos nós das B+Trees foi mantida constante na implementação.

Também foi observado que a taxa aproximada de ocupação caiu de 71,31% para 48,77%. Isso evidencia que aumentar o tamanho físico do bloco não garante, por si só, melhor aproveitamento do espaço quando a organização mantém uma grande quantidade de buckets previamente reservados.

De forma geral, os resultados mostram que não existe uma única estrutura de acesso ideal para todas as consultas. O hashing é especialmente eficiente para buscas pontuais pela chave utilizada na função de dispersão, enquanto a B+Tree oferece maior flexibilidade para índices secundários e consultas por intervalo. Os experimentos também mostram que o tamanho do bloco deve ser analisado em conjunto com a organização lógica das estruturas utilizadas.